

# Object oriented programming OOP

# Object oriented programming

- Characteristics:
  - Subdivision of code into functional units (classes).
  - Creation (instantiation) of independent objects from the classes
  - Referencing dependent objects within variables
- Advantages:
  - Reusable code
  - Better structuring and readability
  - Increased security through access modifiers

# Namespaces and classes

## Namespace

- is used to organize classes and methods
- can be nested
- simplified access to code via using statements

## Class

- describes the structure of objects
  - several objects can be created from one class
  - consist of
    - fields (member variables)
    - properties
    - functions (methods)
    - constructor/destructor

# Fields

- fields are variables of any type within a class
- fields should generally only be private and not externally available
- fields should be accessed via "get" and "set" methods
- usually "store" the data to be accessed

```
public class Person
{
    private string firstname;

    public void SetFirstname(string firstname)
    {
        this.firstname = firstname;
    }

    public string GetFirstname()
    {
        return firstname;
    }
}
```

# Properties

- simplify the creation of the "Get" and "Set" methods

```
private string firstname;

public void SetFirstname(string firstname)
{
    this.firstname = firstname;
}

public string GetFirstname()
{
    return firstname;
}
```

```
//Version 1 without private variable
public string Firstname { get; set; }
```

```
//Version 2 with private variable
private string firstname;
```

```
public string Firstname
{
    get { return firstname; }
    set { firstname = value; }
}
```

# Eigenschaften (Properties)

//Can be get/set from anywhere

```
public string Firstname { get; set; }
```

//Can only be set in this class (not from the outside)

```
public string Lastname { get; private set; }
```

//Value can only be obtained from inside this class

```
public string Secret { private get; set; }
```

```
private string firstname;
```

```
public string Firstname
```

```
{
```

```
    get
```

```
    {
```

```
        return firstname;
```

```
    }
```

```
    set
```

```
    {
```

```
        if (value.Length > 2 && value.Length < 15)
            firstname = value;
```

```
    }
```

```
}
```

# Default values

- Assign default value to a field

```
public class Person
{
    private string firstname = "Max";

    public string Firstname
    {
        get { return firstname; }
        set { firstname = value; }
    }
}
```

# Default values

- Assign default value to an auto property

```
public string Firstname { get; set; } = "Max";
```

- Assign default value at object creation

```
public string Firstname { get; set; }
```

```
public Person()  
{  
    Firstname = "Max";  
}
```



# Constructor

- defines the initial state of the object after initialization
- a class can have several constructors (different parameters)
- if no constructor is specified, a default parameterless constructor is automatically created (e.g. "Person()")

```
private string firstname;  
private string lastname;  
private DateTime birthday;
```

```
public Person(string firstname, string lastname)  
{  
    this.firstname = firstname;  
    this.lastname = lastname;  
}
```

```
public Person(string firstname, string lastname, DateTime birthday) : this(firstname, lastname)  
{  
    this.birthday = birthday;  
}
```

# Objects (instances)

- an object is a block of memory that is built after a class is designed
- there can be several objects of a class
- objects are created with the keyword "new" and a function of the class name (constructor)

```
Person person = new Person("Max", "Test");
```